Row-Partitioned Multithreaded Matrix Multiplication: Benchmarks Report

Approach

The program multiplies two matrices C = A × B using both a sequential and a multithreaded approach. The multithreaded version uses row partitioning: each thread is assigned a disjoint block of rows from A and computes the corresponding rows of C. This avoids locking since each thread writes to unique rows. To improve cache efficiency, the program also precomputes a column-wise representation of B to speed up dot products. Inner loops are partially unrolled for performance.

Benchmark Results

Size (n x n)	Standard (µs)	Multithreaded (µs)	Speedup (×)
64×64	3043	3078	0.99
96×96	8985	5963	1.51
128×128	21903	11616	1.89
160×160	41679	21544	1.93
192×192	111610	34911	3.2

Analysis

The standard version was faster at very small sizes (e.g., 64×64), because the overhead of spawning threads and building cache-friendly structures outweighed the parallel benefit. Starting from 96×96, the multithreaded version achieved clear speedups (up to about 3.2× in these tests). The standard sequential formula can sometimes outperform parallel execution for small matrices due to:

1. Thread creation and scheduling overhead.
2. Extra preprocessing of B into column form.
3. Better cache utilization when the entire matrix fits in cache.

For larger matrices, the cubic growth ($O(n^3)$) of operations ensures the work dominates the fixed overhead, making multithreading advantageous.